

REACT JS...

WHAT IS REACT JS?

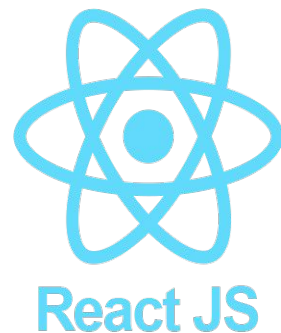
A JavaScript Library for Building User Interfaces

React is an **open-source JavaScript library**

Maintained by **Meta (Facebook)**

Focuses on **component-based UI development**

Mainly used for **Single Page Applications (SPAs)**



HISTORY OF REACT

Created by **Jordan Walke** at Facebook

First released in **2013**

Solved problems of: **Complex UI updates and Poor performance in large apps**

Used in **Facebook ,instagram and whatsapp web**

Became industry standard for frontend development

KEY FEATURES OF REACT

Component-based architecture

JSX syntax

Virtual DOM

Reusable UI components

Strong ecosystem

COMPONENTS IN REACT

Two types

1. Functional Components (modern)
2. Class Components (older)

Each component Has its own logic
And also they Can manage its own state

JSX (JAVAScript XML)

JSX allows writing **HTML inside JavaScript**

Makes UI code more readable

Compiled to JavaScript by Babel

Example:

```
<h1>Hello React</h1>
```

VIRTUAL DOM

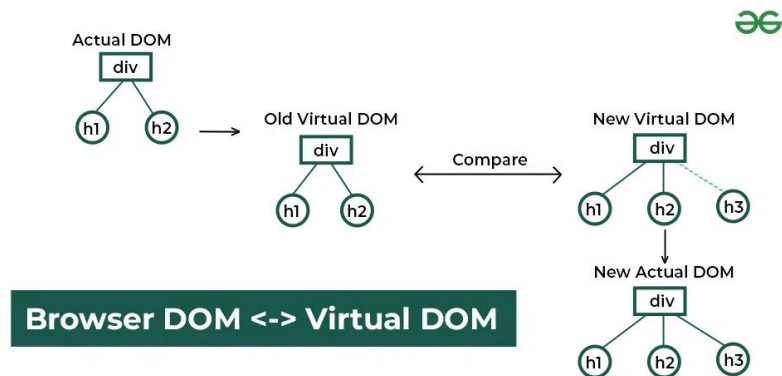
Virtual DOM is a **lightweight copy of the real DOM**

React updates the Virtual DOM first

Finds the **minimum changes needed**

Updates only the changed parts in the real DOM

Improves **performance and speed**



REACT DOM & RENDERING

React DOM connects React with the browser DOM

Uses `createRoot()` to render components

Example:

```
createRoot(root).render(<App />);
```

Efficient rendering using Virtual DOM

Updates only changed UI parts

SINGLE PAGE APPLICATION (SPA)

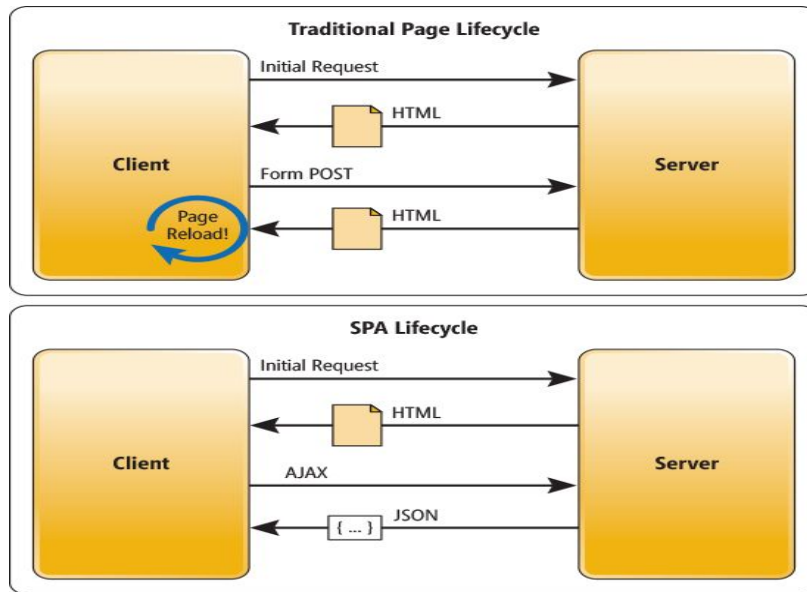
SPA loads **one HTML page**

Content updates without page reload

Faster user experience

Navigation handled using:
React Router

Common in modern web apps



HOOKS & STATE

What are Hooks?

- Hooks are **built-in functions** in React
- They allow functional components to do more things (ex: store data, run side-effects)
- Earlier, only **class components** could do these — Hooks replace that need

What is State?

- State is like the **memory** of a component
- Used to store values that can **change**
 - Example: text input value, counter number, login status, API response
- When state changes → **screen updates automatically**

Common Hooks

- **useState()** → store and update values
- **useEffect()** → run code when screen loads or data changes (fetch API, timers)
- **useRef()** → store a value without re-rendering
- **useContext()** → share data between multiple components

Why are Hooks & State useful?

- Help you make **interactive apps**
- Allow UI to react to **user actions**
- Easy to manage **data and component lifecycle**
- Makes coding **simpler and cleaner**

USESTATE HOOK

Used to manage **component state**

State changes cause re-render

Example:

```
const [count, setCount] = useState(0)
```

count → current state

setCount → updates state

USESTATE HOOK

Handles **side effects**

Runs after component renders

Used for API Calls, Timers, Subscriptions

Example:

```
useEffect(() => {  
    fetchData();  
}, []);
```

USECONTEXT HOOK

Used for **global state management**

Avoids prop drilling

Common use cases

- **Auth data**
- **Theme**
- **Language**

Shares data across components easily

HOW REACT WORKS

UI changes trigger state updates

Virtual DOM updated

React compares old vs new DOM

Applies minimal changes

Updates real DOM efficiently

RECONCILIATION

Process of comparing:

- . Old Virtual DOM
- . New Virtual DOM

Determines what changed

Improves rendering performance

REACT FIBER

New reconciliation engine

Enables The Concurrent rendering Priority-based updates

Improves The Animation and Responsiveness

Makes React apps smoother

WHY USE REACT IN 2025

High demand in job market

Used by top tech companies

Strong community support

Excellent performance

Works with both web and mobile

REST API INTEGRATION IN REACT

Used to connect frontend with backend

Common methods :Fetch(),Axios()

Supports: GET,POST,PUT,DELETE,PATCH

Used inside useEffect()

FETCH VS AXIOS

Fetch

Built-in

More boilerplate

Manual error handling

Axios

Third-party library

cleaner Syntax

Automatic JSON Handling

ADVANTAGES OF REACT

Fast rendering

Reusable components

Easy maintenance

SEO-friendly (with SSR)

Strong ecosystem

Backed by Meta

REACT ECOSYSTEM

React Router – navigation

Redux / Context – state management

Axios – API calls

Next.js – SSR & SEO

React Native – mobile apps

Vite / CRA – project setup

HOW REACT CONNECTS TO SERVER

1. User interacts with UI
2. React updates state
3. API request sent to backend
4. Backend processes request
5. Response returned as JSON
6. React updates UI dynamically

REAL-WORLD EXAMPLE

1. React UI → Employee List
2. Backend → REST API (Node/Express)
3. Axios → Fetch employee data
4. State → Store employees
5. UI → Display dynamically

CONCLUSION

React simplifies UI development

Efficient, scalable, and fast

Widely used in modern applications

Essential skill for frontend developers